

Assignment No: 08 Data Analytics I Create a Linear Regression Model using Python/R to predict home prices using Boston Housing Dataset (<https://www.kaggle.com/c/boston-housing>). The Boston Housing dataset contains information about various houses in Boston through different parameters. There are 506 samples and 14 feature variables in this

In [2]:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

In [3]:

```
df = pd.read_csv('data boston housing.csv')
```

In [4]:

```
df.head()
```

Out[4]:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

In [5]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   crim        506 non-null    float64
 1   zn          506 non-null    float64
 2   indus       506 non-null    float64
 3   chas        506 non-null    int64
 4   nox         506 non-null    float64
 5   rm          506 non-null    float64
 6   age        506 non-null    float64
 7   dis         506 non-null    float64
 8   rad         506 non-null    int64
 9   tax         506 non-null    int64
10   ptratio     506 non-null    float64
11   b           506 non-null    float64
12   lstat       506 non-null    float64
13   medv        506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
```

In [6]:

```
df.describe()
```

Out[6]:

[illegible]

	count	crim	zn	indus	chas	nox	rm	age	dis	rad	tax
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901	3.795043	9.549407	408.2371	711.0000
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861	2.105710	8.707259	168.5371	168.5371
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000	1.129600	1.000000	187.0000	187.0000
25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025000	2.100175	4.000000	279.0000	279.0000
50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500000	3.207450	5.000000	330.0000	330.0000
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075000	5.188425	24.000000	666.0000	666.0000
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000	12.126500	24.000000	711.0000	711.0000

In [7]:

```
df.isnull().sum()
```

Out[7]:

```
crim      0
zn        0
indus     0
chas      0
nox       0
rm        0
age       0
dis       0
rad       0
tax       0
ptratio   0
b         0
lstat     0
medv      0
dtype: int64
```

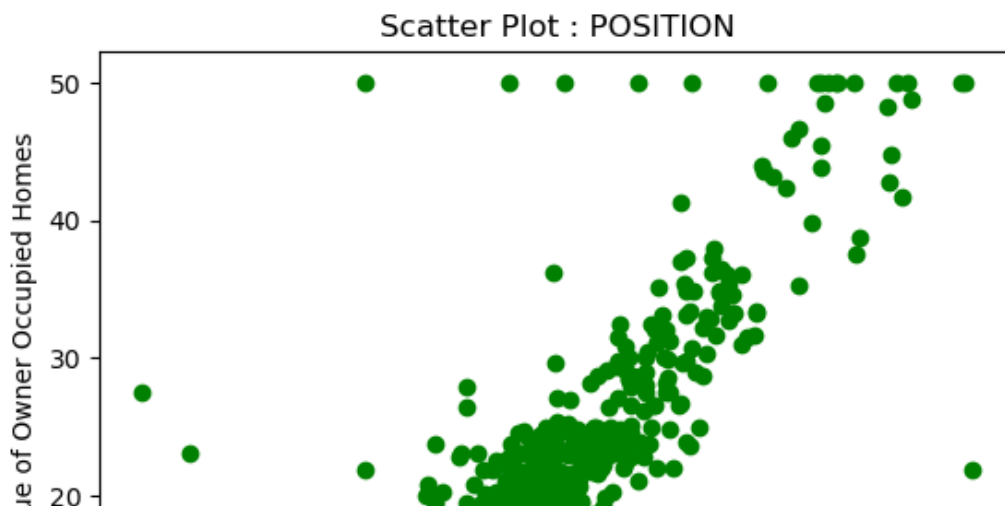
In [8]:

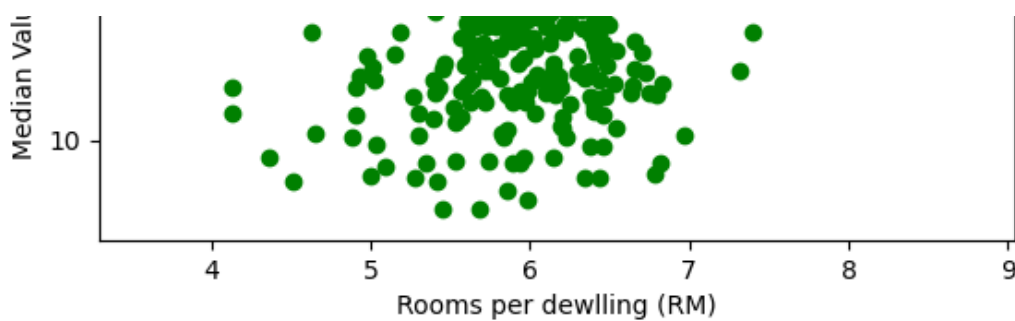
```
df = df.fillna(df.mean())
```

Sample Viva Answer: “The scatter plot visualizes the relationship between the number of rooms per dwelling (rm) and the median house value (medv). Each green point represents a data entry. By plotting this, we can observe whether there is a positive trend — that is, as the number of rooms increases, the median value of the home tends to rise. This kind of visualization is useful for understanding patterns and justifying the use of regression models.”

In [13]:

```
plt.scatter(df['rm'], df['medv'], color="green")
plt.xlabel("Rooms per dwelling (RM)")
plt.ylabel('Median Value of Owner Occupied Homes')
plt.title('Scatter Plot : POSITION')
plt.show()
```





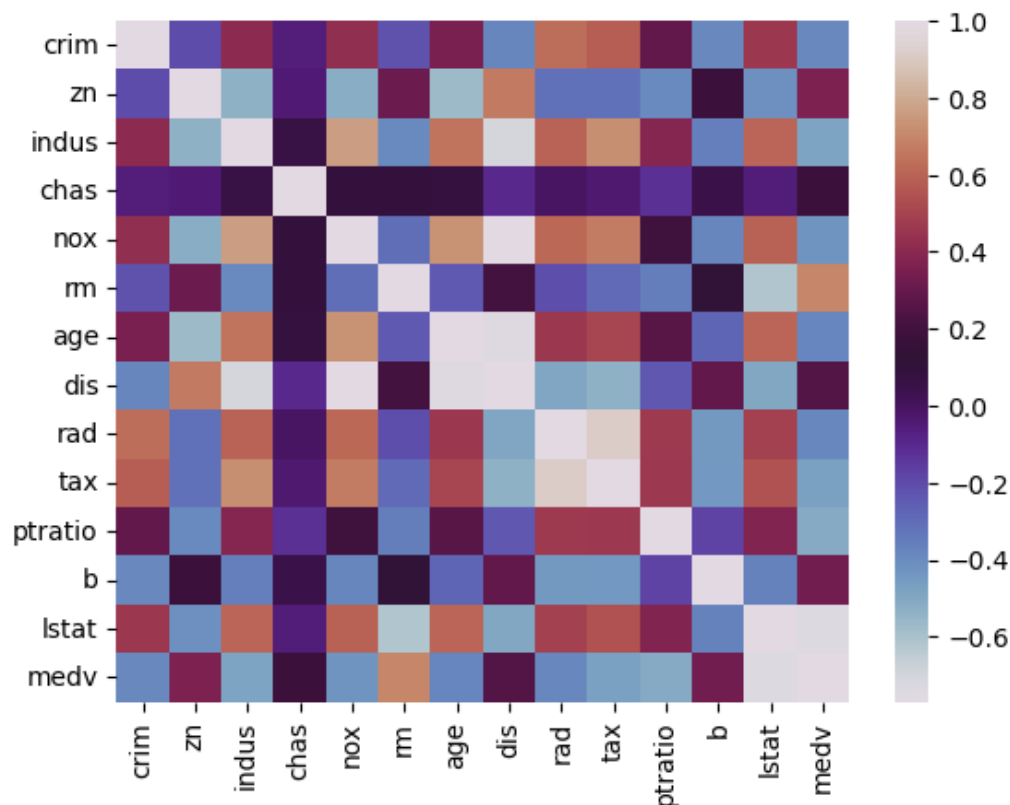
□ **Sample Viva Answer:** “This code creates a heatmap that visually represents the correlation matrix of the dataset. It helps us understand how strongly each feature is linearly related to others. The color map twilight highlights both positive and negative correlations clearly. We use this visualization during exploratory data analysis to detect patterns, such as multicollinearity or strong predictive relationships between variables.”

In [14]:

```
sns.heatmap(df.corr(), cmap="twilight")
```

Out[14]:

<Axes: >



In [15]:

```
X = df.iloc[:,0:-1]
Y = df.iloc[:, -1]
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.30, random_state=42)
```

In [16]:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
```

In [18]:

```
from sklearn.linear_model import LinearRegression
```

```
Lr = LinearRegression()

Lr.fit(X_train, Y_train)
```

Out[18]:

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

In [19]:

```
pred = Lr.predict(X_test)
pred
```

Out[19]:

```
array([30.77834822, 38.66581541, 16.96705276, 26.79035818, 20.36068008,
       24.89386356, 18.98948959, 15.44554414, 24.50205701, 22.12455513,
       27.16709892, 20.60218969, -4.91312377, 23.45520373, 20.44300739,
       27.97454642, 21.82481134,  6.80793603, 43.13876668, 19.08804672,
       28.93737925, 31.91943808, 12.05148619, 25.41493356, 19.60614164,
       17.23543021, 24.61263361, 16.907528  , 24.25623469, 20.73183868,
       23.98650325, 26.66317526, 27.45468242, 19.57475267, 18.15660164,
       19.94422395, 33.04896681, 21.26475982, 25.83570427, 26.36436267,
       15.22545112, 33.61610719, 45.24385042, 18.83620774, 28.91584593,
       18.49470905, 14.91054896, 27.65199578, 21.75498681, 32.36964545,
       23.36168054, 36.5955736 , 16.82098601, 27.59874838, 42.39261672,
       24.39945725, 20.42785358, 34.46608842, 26.52068304, 14.0304141 ,
       24.03170468, 32.40960974, 33.59067192, 17.33583061, 22.46180876,
       18.28308242, 22.01801285, 27.57486429, 32.54366102, 13.95067422,
       21.96577449, 29.21079363, 12.65101384, 17.15704754, 25.56951945,
        6.99478235, 22.73334313, 43.93983367, 20.03571324, 10.919632  ,
       22.50309934, 14.46771561, 23.07376266, 10.52688537, 24.71136556,
       34.38262811, 21.02382869, 27.04400189, 30.94450114, 21.54068028,
       27.48732329,  7.31708375, 21.56254771, 16.57285469, 14.43338806,
       22.26113509, 26.42907376,  1.28686278, 15.98179144, 18.13930581,
       23.62097091, 26.3208465 , 12.12867204, 20.66733158, 25.32719152,
       14.23008516, 19.48076211, 26.9289352 , 22.02461937, 26.11181168,
        9.83654844, 21.30376854, 23.13155127, 28.99045653, 34.46072055,
       17.31422502, 36.81803955, 14.0639201 , 22.38719669, 30.47720506,
       16.96724187, 26.23079837,  6.30719991, 25.6891189 , 27.44337177,
       24.41723783, 26.72754089, 35.52652858, 23.8342233 , 41.30725508,
       15.51580837, 27.19720138, 18.99437335, 22.50080685, 11.95557604,
       23.13463621, 23.4954008 , 34.27666272, 33.53948706, 16.55984176,
       17.98511069, 31.00757978, 26.51337445, 18.4765867 ,  7.99167823,
       28.32517448, 26.1665569 , 19.032183  , 14.47502635, 42.88762567,
       18.69947012, 19.82530166])
```

□ **Explanation for Viva:** This code is used to perform cross-validation for evaluating the performance of a linear regression model (Lr), specifically by calculating the Mean Squared Error (MSE) across multiple folds.

In [20]:

```
from sklearn.model_selection import cross_val_score
MSE = cross_val_score(Lr, X_train, Y_train, scoring='neg_mean_squared_error', cv=10)
np.mean(MSE)
```

Out[20]:

-25.550660791660796

□ **Explanation for Viva:** This code calculates the R-squared (R^2) score for a regression model's predictions, which is a common metric used to evaluate the performance of regression models.

In [21]:

```
from sklearn.metrics import r2_score
score= r2_score(pred, Y_test)
score
```

Out[21]:

0.6455498891708304

In []: